Programovanie v operačných systémoch

06 - Processes, threads

Jozef Šiška



Department of Applied Informatics
Comenius University in Bratislava

2026/2027

Process

Thread

Inter-process communication

Process

- instance of a program
- processor state, context (registers,...), stack
- virtual memory
- resources (file desc., security info (user, group, capabilities))

- `fork()` (+ `exec(...)`) `CreateProcess`
- `exit()`
- `waitpid()`

Thread

- lightweight
- processor state, context (registers,...), stack
- shared virtual memory, resources

- `pthread_create()` (`clone(...)`)
- `pthread_exit()`
- `pthread_join()`

Example

```c
void *task_hello(void *data)
{
    printf("Hello world, data: %p\n", data);
    pthread_exit(NULL);
}

int main (int argc, char *argv[])
{
    pthread_t threads[NUM_THREADS];
    int ret, i;
    for (i = 0; i < NUM_THREADS; ++i) {
        printf("main: pthread_create %d\n", i);
        if ((ret = pthread_create(&threads[i], NULL, task_hello, NULL))) {
            printf("pthread_create: error: %d\n", ret);
            exit(EXIT_FAILURE);
        }
    }

    for (i = 0; i < NUM_THREADS; ++i) {
        void *retval = NULL;
        if ((ret = pthread_join(threads[i], &retval) != 0)) {
            printf("pthread_join: error %d\n", ret);
        } else {
            printf("thread %d finished with return value %p\n", i, retval);
        }
    }
    pthread_exit(NULL);
}
```

Comparison

	process	thread
create	`fork()`	`pthread_create()`
exit	`exit(int status)`	`pthread_exit(void *retval)`
wait	`waitpid()`	`pthread_join()`
identification	unique PID	shared PID unique TID
stack	unique	
signals	shared*	
memory	unique (copy on write)	shared
file descriptors		
mutexes	unique/copy	shared
...		

Inter-process communication (IPC)

- signals — `signal(7)`
  - no data, hard to use
- shared memory — `shm_overview(7)`
  - only data, no events / change notification
  - needs synchronization
- (POSIX | System-V) message queues — `mq_overview(7)`
- pipes / FIFOs — `pipe(7)`
  - single unidirectional stream
  - can be shared by multiple writers / readers (each byte is read once)
- sockets (local, network) — `socket(7)`
  - one/many to one/many, depending on the type

Signals

- Sending

```
int kill(pid_t pid, int sig)
```

- Receiving

```
void handler1(int sig);
sighandler_t signal(int signum, sighandler_t handler);

void handler2(int sig, siginfo_t *siginfo, void *context);
int sigaction(int signum, const struct sigaction *new,
              struct sigaction *old);
```

  Due to the way signal handlers interrupt normal execution of a thread, they usually can't access any complex structures or use synchronization mechanisms: can't call async-signal-safe function, see `signal-safety(7)`. (On linux, check `signalfd(2)`.)

- Blocking

```
int sigprocmask(int how, const sigset_t *set, sigset_t*oldset);
int pthread_sigmask(int how, const sigset_t *set, sigset_t*oldset);
```

  In a multithreaded program, handlers run in an arbitrary thread whose mask allows it. A thread inherits its parent's mask.
  Note that use of `sigprocmask` in multithreaded programs is unspecified.

- Default action for some signals is to terminate the receiving process!

`signal(7)`

Shared memory

```
int open(const char *pathname, int flags, mode_t mode);
int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
int munmap(void *addr, size_t length);
```

- use shm_open to open an existing or create a new shared memory object identified by name
- use mmap to map it into process memory space
- alternatively use mmap on regular files opened by open to get "persistent" filesystem backed shared memory

shm_overview(7)